The Unreasonable Effectiveness of Data for Recommender Systems

Youssef Abdou
youssef.abdou@student.uni-siegen.de
University of Siegen
Siegen, Germany

Abstract

In recommender systems, collecting, storing, and processing large-scale interaction data is increasingly costly in terms of time, energy, and computation, yet it remains unclear when additional data stops providing meaningful gains. This paper investigates how offline recommendation performance evolves as the size of the training dataset increases and whether a saturation point can be observed. We implemented a reproducible Python evaluation workflow with two established toolkits, `LensKit` and `RecBole`, included 11 large public datasets with at least 7 million interactions, and evaluated 10 tool–algorithm combinations. Using absolute stratified user sampling, we trained models on nine sample sizes from 100,000 to 100,000,000 interactions and measured *NDCG*@10. Overall, raw *NDCG* usually increased with sample size, with no observable saturation point. To make result groups comparable, we applied min–max normalization within each group, revealing a clear positive trend in which around 75% of the points at the largest completed sample size also achieved the group's best observed performance. A late-stage slope analysis over the final 10–30% of each group further supported this upward trend: the interquartile range remained entirely non-negative with a median near 1.0. In summary, for traditional recommender systems on typical user–item interaction data, incorporating more training data remains primarily beneficial, while weaker scaling behavior is concentrated in atypical dataset cases and in the algorithmic outlier `RecBole BPR` under our setup.

CCS Concepts

• **Computing methodologies** → **Ranking**; *Machine learning*.

Keywords

Recommender, RecSys, Dataset, Size, Performance, NDCG, Sampling

1 Introduction

1.1 Background

Recommender systems rely on a limited number of parameters, with the dataset size being significant but practically controllable. This work investigates the relationship between dataset size and system performance to guide researchers and practitioners in making informed design choices aligned with their objectives.

1.2 Research Problem

Since large volumes of high-quality, usable data are difficult to gather, the value of relying on ever-larger datasets should be clearly justified. Researchers have highlighted challenges such as scalability, data sparsity, and privacy concerns across domains like e-commerce and media streaming [12]. At the same time, industrial recommender systems must balance potential performance gains against system cost and latency [20].

Related evidence from practice suggests that non-algorithmic factors can dominate measured performance: changing the GUI has been shown to boost CTR by as much as 66%, potentially exceeding gains from algorithm tuning [4]. This influence is therefore a key confounder when conducting experiments on most standard-scale RecSys datasets. However, larger datasets can be more GUI-diverse and might therefore mitigate potential bias, since they are often accumulated over longer timelines during which the interface likely evolves.

Data pruning has on one hand been shown to result in up to a 3% increase in performance when detrimental users are excluded [11], while on the other hand a broader empirical study concluded that pruning should generally be avoided whenever possible [3]. Resolving this discrepancy in settings that incorporate larger data volumes remains an open question.

Model training, particularly on large datasets, is computationally expensive and resource-intensive. Our longest observed cases occurred among LensKit runs on `iPinYou` at the 1*m* scale, where every LensKit run required more than 20 hours of end-to-end runtime. We observed similarly long end-to-end runtimes across different libraries and datasets. Researchers describe deep learning model training as especially time-consuming on large datasets, which can take up to several months [6].

Recent work by Vente et al. [17] emphasizes the considerable environmental cost of research on recommender systems, especially when using deep learning and large-scale datasets. Their analysis of full papers from the 2023 ACM Recommender Systems Conference shows that the average study consumed approximately 6,854 kWh of electricity, and that deep learning models required about eight times more energy than traditional machine learning methods without demonstrating clear performance benefits. The study further reports that the carbon footprint of recommender-system experiments has increased dramatically over the past decade, highlighting how larger datasets and more complex models intensify energy consumption and CO_2 emissions.

Consequently, the pursuit of larger datasets must be critically assessed; the environmental and computational costs require that the need for such data abundance be clearly justified.

1.3 Research Question

How does the performance of recommender systems evolve as the size of the training dataset increases? In particular, does performance improve steadily with more data or does it reach a point of diminishing returns? Is there an optimal dataset size at which a given recommender system likely achieves its best performance, beyond which additional training data yields no significant improvement? If such a saturation point exists, it needs to be identified, as recognizing it would greatly help reduce unnecessary computational costs and energy consumption.

1.4 Research Hypothesis

We hypothesize that, within recommender systems, predictive performance continues to improve as training data grows across the scales studied, rather than reaching a clear saturation point. This expectation is consistent with prior work showing that larger datasets often yield higher accuracy and more stable results [1], and with the broader view that scale can have an "unreasonable" positive effect [9]. At the same time, earlier studies show that the apparent benefit of additional data can depend on the model and evaluation design [5, 16]. Determining whether saturation occurs is therefore crucial: evidence of saturation would shift attention toward understanding bottlenecks, whereas evidence against it would justify further investment in efficient large-scale data collection.

2 Methodology

2.1 Overview

We created a standard Python project that includes the libraries for two established tools, `LensKit` [7] and `RecBole` [19], as well as other necessary utility libraries. We gathered freely available datasets from a list [13, 14] maintained by the `RecBole` team, prioritizing the larger options. An experimental run generally consists of multiple iterations as follows: first, load a prepared dataset; sample it to a specified number of rows using a chosen sampling strategy; select an algorithm from the list associated with the corresponding tool; provide the sampled dataset to the tool along with the selected algorithm; extract the resulting performance metric value; and finally, record it in the results file. The implementation and experiment scripts are publicly available [15].

2.2 Environment

Python 3.11 and pinned library versions were required for compatibility across the workflow. These explicit versions are kept in the requirements file. The main utility libraries used were NumPy, pandas, and Matplotlib, while the selected recommender frameworks were `LensKit` 2025.2.0 and `RecBole` 1.2.1. The project is configured for SLURM-based [18] execution and includes scripts for running jobs sequentially and in parallel, a Singularity [10] container definition, and a Makefile [8]. The OMNI HPC cluster of the University of Siegen was used for most computationally intensive jobs, where GPU-equipped nodes made the full experiment suite practical.

2.3 Configurations

Nearly all configurations are defined in the constants module. This includes settings shared between all `RecBole` algorithms, `RecBole` algorithm-specific settings, and `LensKit` settings. Hyperparameters and environment-related values are also handled in this module. In general, values were chosen to reduce runtime while remaining sufficient for proper training, since training computationally expensive algorithms on large datasets is not always practically feasible.

2.4 Datasets

Datasets from the list [13, 14] previously mentioned in the overview subsection were selected with a clear preference for those with the largest interaction counts. Explicit interactions were generally preferred, since some algorithms do not work otherwise; therefore, most included datasets had a ratings column or an equivalent. All included datasets contained at least 7 million interactions. Various forms of data cleaning were applied: the timestamp columns were uniformly removed and the uniqueness of the user ID–item ID pairs was enforced. Depending on the type of information, duplicates were either aggregated by summation or dropped so that only one record remained. In addition, dtypes were downcast to the lowest viable choice, and the datasets were stored as parquet [2] files. Consequently, the loading time and the required storage capacity were considerably reduced.

2.5 Sampling

Differing sampling strategies were available as part of the experiment's run mode. First, the size specification method (sizing, for simplicity) indicates whether the dataset is to be sampled to a fraction (percentage) or to an absolute number of rows. Second, the sample selection strategy can be one of four options: `Random Sampling`, `Stratified User Sampling`, `Stratified Item Sampling`, and `Stratified Hybrid Sampling`. `Random Sampling` is seeded and therefore deterministic, which suits the purpose of the experiment. All stratified variants aim to preserve representation ratios across groups, such that no group is over- or under-represented in the resulting sample. For example, in `Stratified User Sampling`, a user representing 10% of the full dataset would also represent 10% of the sample. The hybrid option combines the previous two approaches: stratified sampling by item, followed by stratified sampling by user. When targeting an absolute sample size with this strategy, fully preserving group ratios would typically yield a significantly larger sample. To address this, we slightly over-allocate group ratios and then trim the surplus via random sampling to obtain the exact desired size.

2.6 Algorithms

Five algorithms per tool were selected. The `LensKit` scoring models included were PopScorer, ItemKNNScorer, ImplicitMFScorer, BiasedMFScorer, and BiasedSVDScorer. For `RecBole`, the following recommendation models were selected: Pop, ItemKNN, BPR, NeuMF, and SimpleX.

2.7 Experiment Variables

Different values for experiment-related variables are directly configurable. The settings used to obtain our reported results were as follows: Top-10 recommendations for performance evaluation,

absolute sizing, stratified user sampling, and nine sampling sizes ranging from 100 thousand to 100 million.

3 Results

3.1 Terminology

Instance. A result *instance* is the output of a single evaluation run for a specific (tool, algorithm, dataset, sample size) combination. For example, `LensKit-Popularity-MovieLens-25M` $\mapsto$ NDCG $\approx$ 0.17. In Figures 2 and 3, each plotted point (x, y) corresponds to one instance, where x is the sample size and y is the *NDCG* value.

Group. A result *group* is the collection of instances that share the same (tool, algorithm, dataset) combination, varying only in sample size. Equivalently, a group is a set of points $\{(x_i, y_i)\}_{i=1}^{n}$ with fixed tool/algorithm/dataset and different x_i. In Figures 2 and 3, each colored line corresponds to one group.

Min–max normalization (within a group). For a given group g with instances $\{(x_i, y_i)\}_{i=1}^{n}$, we normalize sample size and *NDCG within that group* as

$$x_i' = \frac{x_i - x_{\min}}{x_{\max} - x_{\min}}, \qquad y_i' = \frac{y_i - y_{\min}}{y_{\max} - y_{\min}}.$$

Here $x_{\min}$ and $x_{\max}$ are the minimum and maximum x-values within the group g, and $y_{\min}$ and $y_{\max}$ are the minimum and maximum y-values within the group g.

Late-stage slope. For each min–max normalized group g, we extract a single *late-stage slope* as follows. Let (x_2', y_2') be the point with maximal normalized sample size, i.e. $x_2' = 1$. Choose (x_1', y_1') as the point with the largest x_1' such that

$$0.1 \le x_2' - x_1' \le 0.3.$$

The late-stage slope is then

$$m_g = \frac{y_2' - y_1'}{x_2' - x_1'}.$$

Figure 1: Legend

3.2 NDCG Trends

Performance increases steadily with sample size, with no visible saturation: Figures 2 and 3 show the majority of the raw performance results. Each line plot shows *NDCG* as a function of sampled input dataset size for the corresponding tool–algorithm pair. Within each plot, each colored line represents a result group. Each group contains instances at the sample sizes defined in the *experiment variables*, up to either the full dataset size or the maximum configured sample size ($100m$), whichever is smaller. Some values could not be computed within a reasonable time and are therefore missing. In most plots, the overall trend increases with

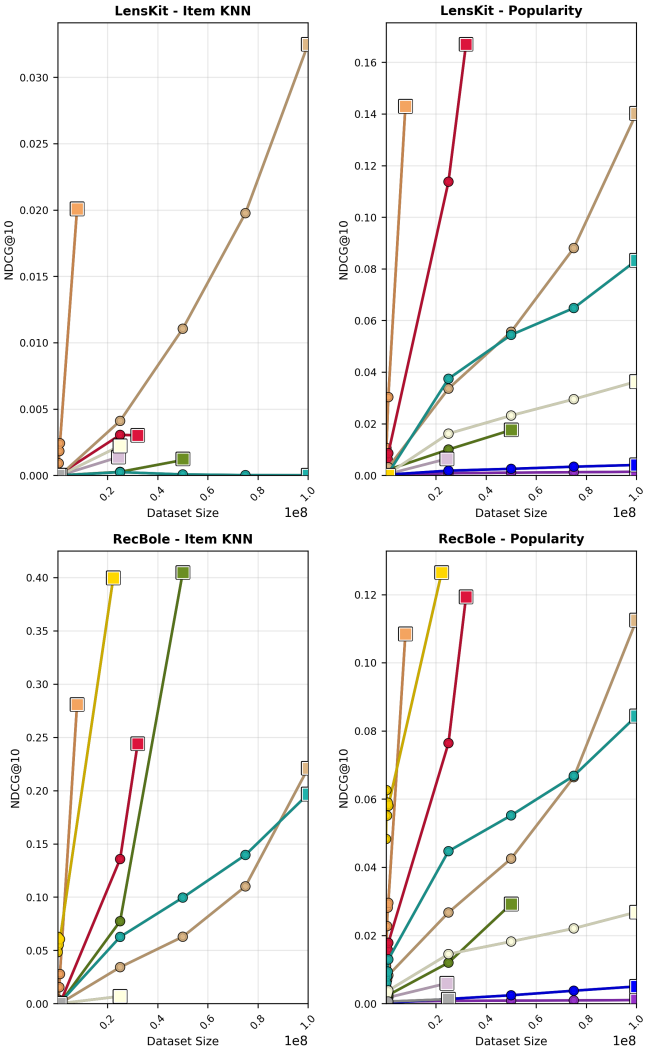

Figure 2: *NDCG*@10 **vs. Sample Size**

sample size, with no visible point at which *NDCG* begins to diminish. Datasets that consistently show both a steep upward trend and a relatively high *NDCG* include `MovieLens` and `Netflix`, reaching approximately 0.25 at the full $32m$ and around 0.22 at $100m$, respectively (both from `RecBole's Item KNN` results). In contrast, `Last.fm` shows no convincing performance improvement with increasing sample size in any group; its highest value is approximately 0.005 when training on $100m$ rows using the `RecBole Popularity` model. `Alibaba-iFashion` shows even lower values, failing to exceed 0.002 in all instances. Among the algorithms, `RecBole BPR` emerges as a clear outlier.

3.3 Normalized Values Scatter

Near-linear upward trend consistent across comparable normalized groups: Figure 4 reduces the gaps that arise when combining result groups with differing value ranges by applying per-group min–max normalization. Both axes are scaled to $[0.0, 1.0]$: on the

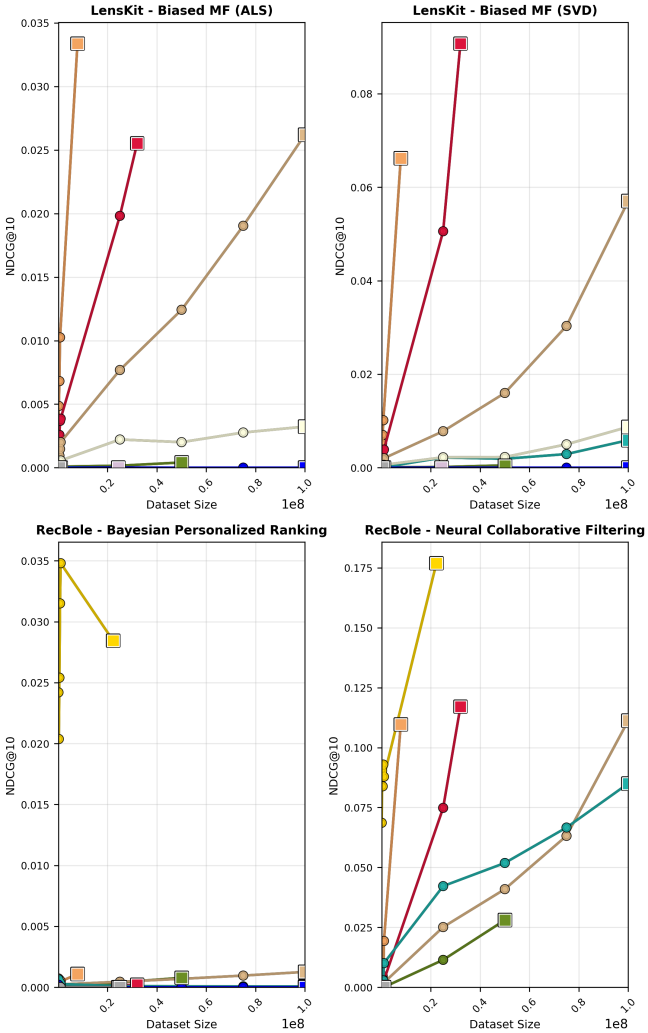

Figure 3: $NDCG@10$ **vs. Sample Size cont.**

x-axis, 1.0 corresponds to the largest sample size within a group and 0.0 to the smallest; the y-axis is scaled in the same way for $NDCG$. A point at $(0.0, 0.0)$ indicates that the lowest $NDCG$ in the group was obtained from the smallest sample size, whereas $(1.0, 1.0)$ indicates that the largest sample produced the highest $NDCG$. Colors distinguish datasets (not groups). For visual clarity, a uniform random jitter in $[-0.0125, 0.0125]$ was applied, since overlap, especially at $(1.0, 1.0)$, was substantial; roughly 75% of points with $x = 1.0$ also had $y = 1.0$, as per Figure 5. This suggests that, for most groups, peak performance occurs at the largest sample size. The region from $(0.0, 0.0)$ to $(0.2, 0.2)$ is densest because many of the sample sizes, defined in the *experiment variables*, lie in the lower portion of each group's sample-size range (between $100k$ and $1m$), which therefore maps to small normalized x values. The dashed diagonal is a reference line for a perfect positive linear relationship. Although clusters near both ends of the line are tightly concentrated, the sparser mid-range points align with the positive trend nonetheless. Outliers are visible across the plot, indicating

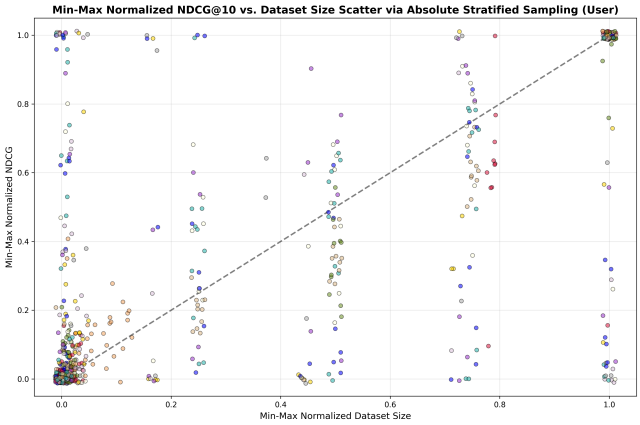

Figure 4: Normalized $NDCG@10$ **vs. Sample Size Scatter**

Scatter Point Distribution via Absolute Stratified Sampling (User)

Region	Count (±0.05)	% (±0.05)	Count (±0.10)	% (±0.10)
(0.00, 0.00)	287	70.0%	328	77.0%
(0.25, 0.25)	9	23.7%	17	32.1%
(0.50, 0.50)	8	16.7%	14	22.2%
(0.75, 0.75)	11	17.5%	19	30.2%
(1.00, 1.00)	82	73.9%	83	74.8%

Total points: 725 | % = count / points in x-band

Figure 5: Figure 4 distributions metadata

that some result groups deviate from this positive pattern; however, this is consistent with the anomalous behaviors observed in the raw $NDCG$ trend plots shown in Figures 2 and 3.

3.4 Late-Stage Slopes Distribution

Late-stage trends suggest increased performance if more data is introduced: Figure 6 shows a box plot that highlights the distribution of late-stage slope values for the normalized results shown in Figure 4, representing the "final rise" or the last observable trend. Each point is the slope over the final $x\%$ of the data, where approximately $10 \leq x \leq 30$. Result groups with fewer than five instances ($n < 5$) were excluded. The late-stage slope indicates whether a likely point of saturation has been reached, suggested by near-zero values that appear near the horizontal red reference line. Conversely, positive values above the reference suggest that adding more data will likely improve performance, pointing to a continued positive relationship that may reasonably be interpreted as truncated by data limitations. A small number of points (exactly 8) lie on the reference line, with slopes approximately equal to 0.0. The lower and upper whiskers are at around -2.25 and 3.5, respectively, with two points falling below the lower whisker but none above the upper. The interquartile range spans around 0.0 to approximately

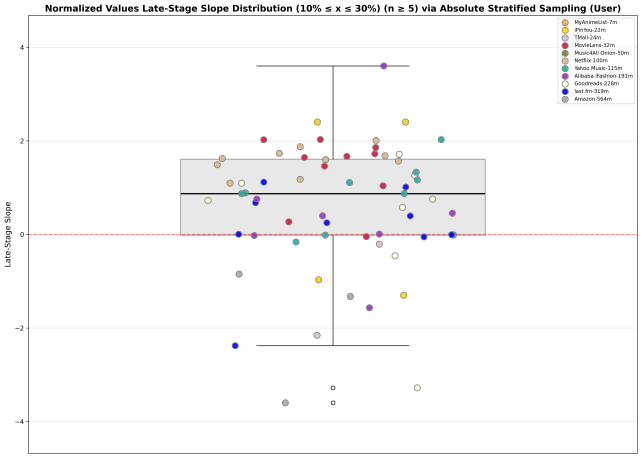

Figure 6: Normalized Values Late-Stage Slope Distribution

1.5, which is entirely non-negative. The reference line coincides with the lower quartile, and the median is nearly 1.0.

4 Conclusion

This study supports the hypothesis that, for general recommender systems trained on standard numerical user–item feedback, $NDCG@10$ scales positively with additional interaction data rather than reaching a clear saturation point within the investigated range and evaluated offline setup. Across 11 large public datasets, 10 tool–algorithm combinations, and sample sizes ranging from 10^5 to 10^8 interactions, recommendation performance usually improves as more interaction data are provided. In many result groups, the largest completed sample yields the best observed $NDCG@10$, while late-stage trends remain predominantly positive. Weaker scaling behavior is concentrated in a limited set of atypical cases, most notably the datasets `Last.fm`, `Alibaba-iFashion`, and `Amazon`, and the algorithm `RecBole BPR`, where dataset-specific feedback characteristics or runtime-constrained configuration choices likely play a role. Taken together, these findings suggest that, for typical user–item interaction data and traditional recommender algorithms under the evaluated setup, additional training data often continues to deliver practically meaningful gains.

References

[1] Ajiboye Abdulraheem, Ruzaini Abdullah Arshah, and Hongwu Qin. 2015. Evaluating the Effect of Dataset Size on Predictive Model Using Supervised Learning Technique. *International Journal of Software Engineering & Computer Sciences (IJSECS)* 1 (02 2015), 75–84. doi:10.15282/ijsecs.1.2015.6.0006

[2] Apache Parquet contributors. [n. d.]. Apache Parquet. https://parquet.apache.org/. Accessed: 2026-03-17.

[3] Joeran Beel and Victor Brunel. 2019. Data pruning in recommender systems research: Best-practice or malpractice. In *13th ACM Conference on Recommender Systems (RecSys)*, Vol. 2431. CEUR-WS, 26–30.

[4] Joeran Beel and Haley Dixon. 2021. The 'Unreasonable' Effectiveness of Graphical User Interfaces for Recommender Systems. In *Adjunct Proceedings of the 29th ACM Conference on User Modeling, Adaptation and Personalization* (Utrecht, Netherlands) *(UMAP '21)*. Association for Computing Machinery, New York, NY, USA, 22–28. doi:10.1145/3450614.3461682

[5] Cagatay Catal and Banu Diri. 2009. Investigating the effect of dataset size, metrics sets, and feature selection techniques on software fault prediction problem. *Information Sciences* 179, 8 (2009), 1040–1058.

[6] Qifei Dong and Gang Luo. 2020. Progress Indication for Deep Learning Model Training: A Feasibility Demonstration. *IEEE Access* 8 (2020), 79811–79843. doi:10.1109/ACCESS.2020.2989684

[7] Michael D. Ekstrand. 2020. LensKit for Python: Next-Generation Software for Recommender Systems Experiments. In *Proceedings of the 29th ACM International Conference on Information and Knowledge Management*. doi:10.1145/3340531.3412778

[8] Stuart I. Feldman. 1979. Make — a program for maintaining computer programs. *Software: Practice and Experience* 9, 4 (1979), 255–265. arXiv:https://onlinelibrary.wiley.com/doi/pdf/10.1002/spe.4380090402 doi:10.1002/spe.4380090402

[9] Alon Halevy, Peter Norvig, and Fernando Pereira. 2009. The unreasonable effectiveness of data. *IEEE intelligent systems* 24, 2 (2009), 8–12.

[10] Gregory M. Kurtzer, Vanessa Sochat, and Michael W. Bauer. 2017. Singularity: Scientific containers for mobility of compute. *PLOS ONE* 12, 5 (05 2017), 1–20. doi:10.1371/journal.pone.0177459

[11] Philipp Meister, Lukas Wegmeth, Tobias Vente, and Joeran Beel. 2024. Removing Bad Influence: Identifying and Pruning Detrimental Users in Collaborative Filtering Recommender Systems.. In *RobustRecSys@ RecSys*. 8–11.

[12] Shaina Raza, Mizanur Rahman, Safiullah Kamawal, Armin Toroghi, Ananya Raval, Farshad Navah, and Amirmohammad Kazemeini. 2026. A comprehensive review of recommender systems: Transitioning from theory to practice. *Computer Science Review* 59 (2026), 100849. doi:10.1016/j.cosrev.2025.100849

[13] RUCAIBox. 2024. Dataset List | RecBole. https://recbole.io/dataset_list.html. Accessed: 2026-03-08.

[14] RUCAIBox. 2024. RecSysDatasets: Public Data Sources for Recommender Systems. https://github.com/RUCAIBox/RecSysDatasets. GitHub repository, accessed: 2026-03-08.

[15] Youssef Tarek Tewfik. 2026. unreasonable-effectiveness-recsys. https://github.com/Youssef-Tarek-Tewfik/unreasonable-effectiveness-recsys. GitHub repository, accessed: 2026-03-19.

[16] Andrius Vabalas, Emma Gowen, Ellen Poliakoff, and Alexander J Casson. 2019. Machine learning algorithm validation with a limited sample size. *PloS one* 14, 11 (2019), e0224365.

[17] Tobias Vente, Lukas Wegmeth, Alan Said, and Joeran Beel. 2024. From Clicks to Carbon: The Environmental Toll of Recommender Systems. In *Proceedings of the 18th ACM Conference on Recommender Systems* (Bari, Italy) *(RecSys '24)*. Association for Computing Machinery, New York, NY, USA, 580–590. doi:10.1145/3640457.3688074

[18] Andy B. Yoo, Morris A. Jette, and Mark Grondona. 2003. SLURM: Simple Linux Utility for Resource Management. In *Job Scheduling Strategies for Parallel Processing*, Dror Feitelson, Larry Rudolph, and Uwe Schwiegelshohn (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 44–60.

[19] Wayne Xin Zhao, Shanlei Mu, Yupeng Hou, Zihan Lin, Yushuo Chen, Xingyu Pan, Kaiyuan Li, Yujie Lu, Hui Wang, Changxin Tian, Yingqian Min, Zhichao Feng, Xinyan Fan, Xu Chen, Pengfei Wang, Wendi Ji, Yaliang Li, Xiaoling Wang, and Ji-Rong Wen. 2021. RecBole: Towards a Unified, Comprehensive and Efficient Framework for Recommendation Algorithms. In *CIKM*. ACM, 4653–4664.

[20] Kuan Zou and Aixin Sun. 2025. A Survey of Real-World Recommender Systems: Challenges, Constraints, and Industrial Perspectives. arXiv:2509.06002 [cs.IR] https://arxiv.org/abs/2509.06002